

GReinSS

Generative Modeling of Discrete Latent Structures via Dynamic Policy Gradients

Stefan Ivanovic · Ge Liu · Mohammed El-Kebir

University of Illinois Urbana-Champaign

NCI Spring School on Algorithmic Cancer Biology — Tutorial (30 min)

The recurring problem in cancer genomics

We rarely observe the biological state we care about. We observe **something generated from it**.

Phylogenies

Tumor evolution trees
(*observed: noisy DNA-seq*)

CNV profiles

Copy-number states per cell
(*observed: read depth*)

RNA isoforms

Spliced transcripts
(*observed: short reads*)

In each case we know the **forward model** $\Pr(X \mid S)$ — how a state S produces data X — but the state S itself is **latent, discrete, and combinatorially large**.

Two things we want

Setup. States $S_1^*, \dots, S_N^* \sim \Pr^*(S)$,
unobserved.

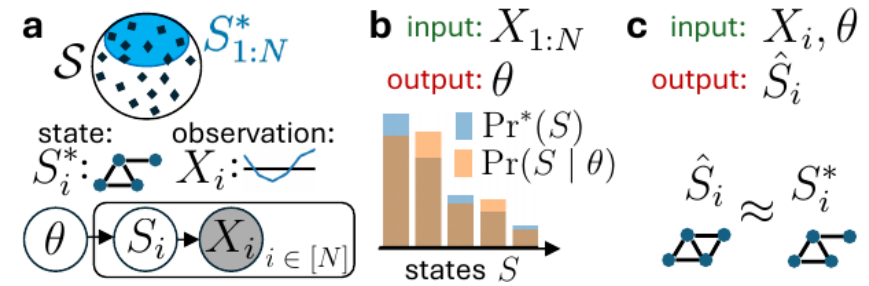
We see indirect observations $X_1, \dots, X_N$, and
can compute $\Pr(X \mid S)$.

We fit a generative model $\Pr(S \mid \theta)$ so that

$$\Pr(X \mid \theta) = \sum_S \Pr(X \mid S) \Pr(S \mid \theta)$$

① **Learning.** Find θ maximizing the data
likelihood

$$\Pr(X_{1:N} \mid \theta) = \prod_i \Pr(X_i \mid \theta)$$



② **Inference.** For each X_i , recover
 $\hat{S}_i = \arg \max_S \Pr(X_i \mid S) \Pr(S \mid \theta)$

Why the usual tools struggle

Approach	Problem
Expectation–Maximization	E-step expectation over $\mathcal{S}$ is intractable when $\mathcal{S}$ is combinatorial
Variational autoencoders	learn <i>artificial</i> continuous latents — not the mechanistic S you want
Local search ($\arg \max_S \Pr(X_i \mid S)$)	ignores the shared model $\Pr(S \mid \theta)$ across observations
Naive policy gradient	collapses to the single highest-reward state
GFlowNets	need known terminal rewards , not a likelihood to maximize

Gap: none of these directly maximize $\Pr(X_{1:N} \mid \theta)$ over a *discrete, combinatorial* state space.

The GReinSS idea

Build the discrete state step by step with a policy, and reward trajectories by their share of each observation's likelihood.

Generative Reinforcement Learning of Structured States

Generate states as trajectories

A **policy** with parameters θ builds a state by a sequence of actions — a **trajectory** τ ending in a terminal state $S(\tau) \in \mathcal{S}$.

- add an **edge** $\rightarrow$ a graph
- add an **element** $\rightarrow$ a set
- add a **junction** $\rightarrow$ an isoform

Sequential construction makes *combinatorial* spaces tractable: no summation over $\mathcal{S}$, just **sampling** trajectories.

$$\Pr(X \mid \theta) = \mathbb{E}_{\tau \sim \Pr(\tau \mid \theta)} [\Pr(X \mid S(\tau))]$$

Estimate by sampling M trajectories:

$$\Pr(X_i \mid \theta) \approx \frac{1}{M} \sum_j \Pr(X_i \mid \tau_j)$$

The one equation that matters

Train with a policy gradient — but with a **dynamically rescaled reward**:

$$r(\tau) = \sum_{i=1}^N \frac{\Pr(X_i \mid \tau)}{\Pr(X_i \mid \theta)}$$

Theorem. The policy gradient $\mathbb{E}_{\tau} \left[r(\tau) \frac{d}{d\theta} \log \Pr(\tau \mid \theta) \right]$ is an **unbiased estimator** of $\frac{d}{d\theta} \log \Pr(X_{1:N} \mid \theta)$.

So standard policy-gradient ascent with this reward = **maximum-likelihood** learning of $\Pr(S \mid \theta)$.

Why the denominator? (intuition)

The rescaling rewards a trajectory for its **proportional contribution** to $\Pr(X_i \mid \theta)$ — not its raw probability. This makes the policy **spread** over the states the data needs, instead of collapsing to one.

Toy: 3 states, 2 observations.

$$\Pr(X_1|S_1) = .5, \Pr(X_2|S_2) = .3, \Pr(X_2|S_3) = .2$$

Naive PG → chases the max reward (τ_1):

$$\Pr(\tau_1) = 1 \Rightarrow \Pr(X_2 \mid \theta) = 0 \\ \Rightarrow \Pr(X_{1:N} \mid \theta) = \mathbf{0}$$

✗

GReinSS → rewards balance out:

$$\Pr(\tau_1) = \Pr(\tau_2) = 0.5, \Pr(\tau_3) = 0 \\ \Rightarrow \Pr(X_{1:N} \mid \theta) = 0.0375 \checkmark \text{ (optimal)}$$

a	$\Pr(X \mid S)$			b	$S(\tau_i) = S_i$		c	$\uparrow \Pr(\tau_i \mid \theta) \Rightarrow \downarrow r(\tau_i)$	d	$\Pr(\tau_i \mid \theta)$		
	S_1	S_2	S_3		$S(\tau_1) = S_1$	$S(\tau_2) = S_2$				$\Pr(\tau_1 \mid \theta) = 0.5$	$\Pr(\tau_2 \mid \theta) = 0.5$	$\Pr(\tau_3 \mid \theta) = 0$
X_1	0.5	0	0									
X_2	0	0.3	0.2									

The training loop

Repeat until convergence:

1. **Sample** trajectories $\tau \sim \text{Pr}(\tau \mid \theta)$
2. **Score** each with $\text{Pr}(X_i \mid \tau)$
3. **Rewards** $r(\tau) = \sum_i \text{Pr}(X_i \mid \tau) / \text{Pr}(X_i \mid \theta)$
4. **Policy-gradient step** on θ
5. **Recompute** $\text{Pr}(X_i \mid \theta) \rightarrow$ new rewards

You provide only two things:

- a way to **generate** S (grow it action-by-action)
- the likelihood $\text{Pr}(X \mid S)$

Everything else is generic.

Mini-batching over observations keeps it unbiased and scalable (Thm.).

Off-policy learning (when on-policy is too slow)

If sampling $\Pr(\tau \mid \theta)$ rarely produces states that explain any X_i , learning stalls.

Sample where the data says to look — bias toward the Bayes posterior:

$$q(\tau \mid X_{1:N}, \theta) = \frac{1}{N} \sum_i \Pr(\tau \mid X_i, \theta), \quad \Pr(\tau \mid X_i, \theta) = \frac{\Pr(X_i \mid \tau) \Pr(\tau \mid \theta)}{\Pr(X_i \mid \theta)}$$

This proposal is the **unbiased, variance-minimizing** one (Thm.); importance sampling keeps the gradient correct. *In cancer apps, a cheap heuristic (e.g. CNNaive in CNRein) seeds plausible states.*

→ NOTEBOOK · Demo 1: Set reconstruction

Recover binary **sets** from noisy real-valued measurements. *Trains live in ~10 s.*

Problem. $S_i^* \subseteq \mathcal{U}$, observe

$X_{i,j} \sim \mathcal{N}(1, \sigma^2)$ if $j \in S_i^*$, else $\mathcal{N}(0, \sigma^2)$.

You'll write:

```
def log_pr_x_given_g(state, obs):  
    return -0.5*np.sum((obs-state)**2)/sigma**2
```

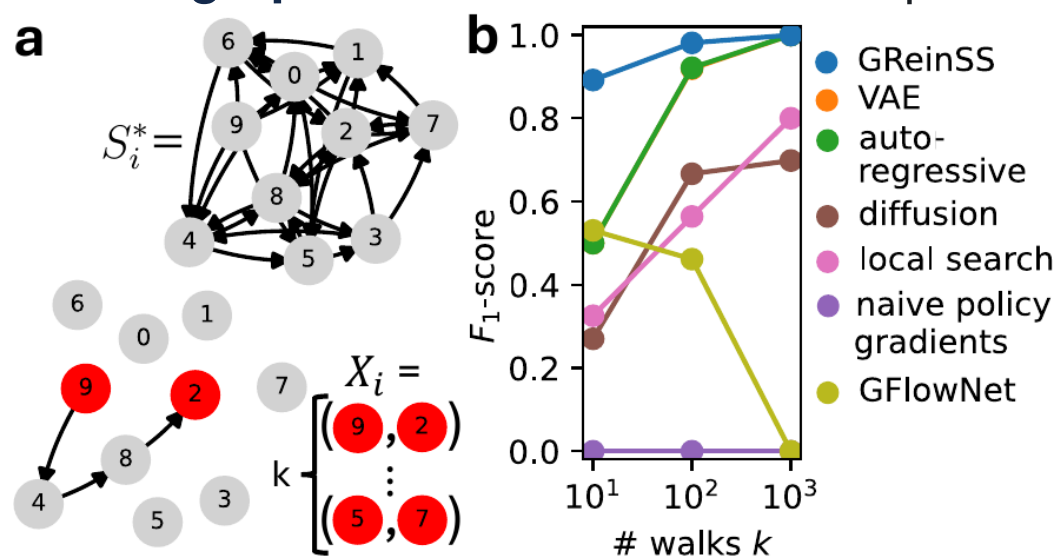
...then `simpleTrainModel(...)`.

Watch for:

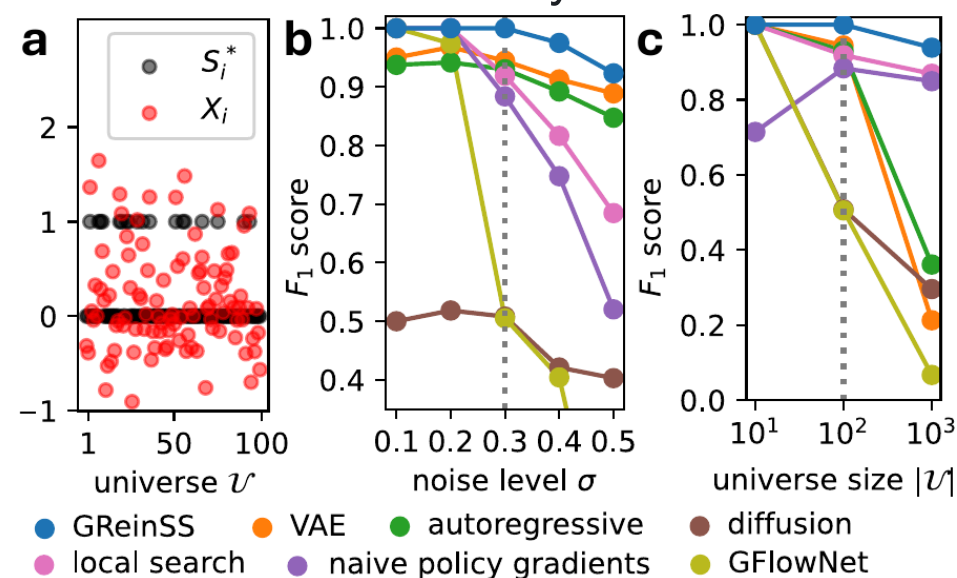
- median $\log \Pr(X_i \mid \theta)$ climbing to ~0
- recovered sets vs. thresholding the noise
- where the **shared model fixes** noisy bits

Results — simulations

Latent graphs from random-walk endpoints



Latent sets from noisy measurements



$k = 10$ walks: GReinSS $F_1 = \mathbf{0.891}$; all baselines < 0.55

$|\mathcal{U}| = 1000$: GReinSS $F_1 = \mathbf{0.938}$; GEM baselines < 0.4

Dynamic rewards are a **small** code change over naive PG — but decisive. Naive PG here predicts the **empty graph** ($F_1 = 0$).

→ NOTEBOOK · Demo 2: Graph inference (pre-trained)

Latent **directed graphs** from start/end points of k absorbing random walks.

State = 90 possible directed edges (10 nodes).

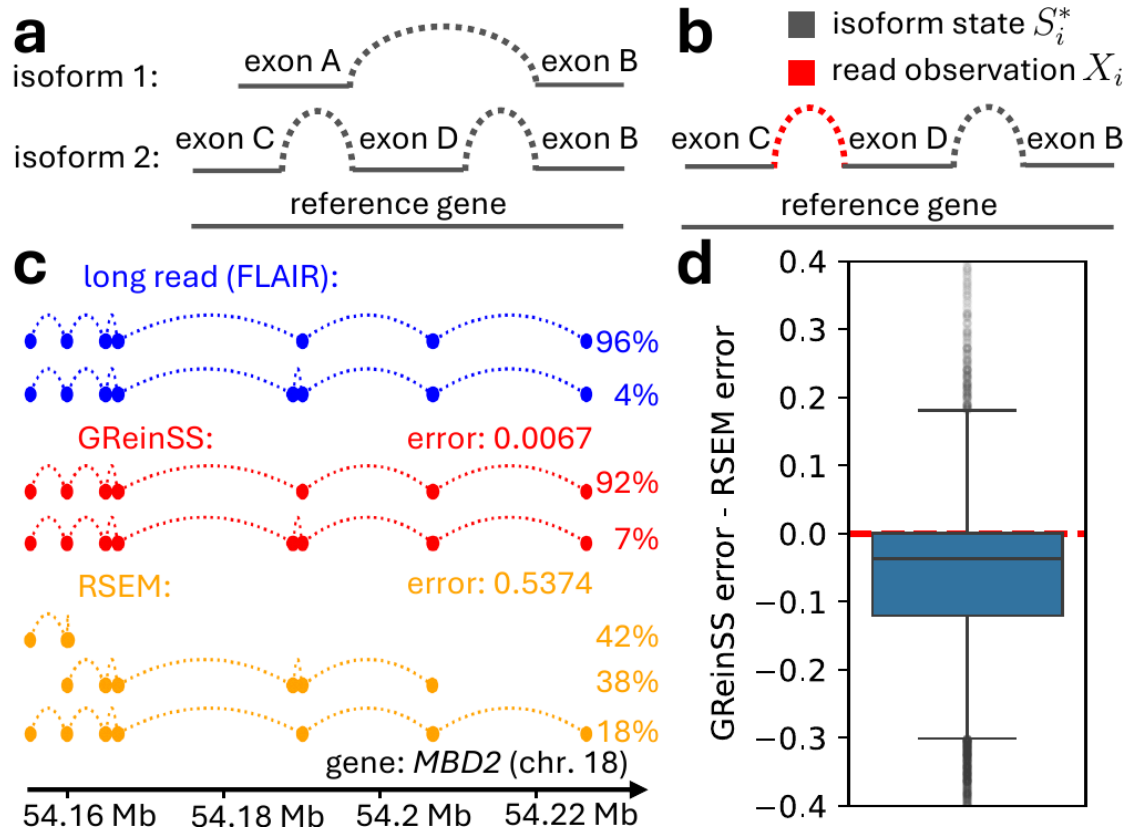
$\Pr(X \mid S)$ from the shifted-Laplacian $(L + I)^{-1}$ random-walk model.

We **load a pre-trained policy**, run inference, and score edge-recovery F_1 against ground-truth graphs.

Watch for:

- training likelihood curve (pre-computed)
- a reconstructed graph $\hat{S}_i$ vs. true S_i^*
- per-graph F_1 distribution

Application — RNA isoforms beat RSEM



State S = an isoform (chosen exon junctions) + sample + read position.

$\Pr(X | S) = 1$ iff read position & sample match — trivial forward model.

On **GTEx** (61 samples w/ matched long reads, 14,390 genes):

- GReinSS **beats RSEM by ≥ 0.05** on **46.6%** of genes; RSEM beats GReinSS on only **9.4%**
- *MBD2* example: GReinSS error **0.007** vs RSEM **0.537**

GReinSS already powers two cancer methods

CloMu

Tumor **phylogenies** of SNVs

States: mutation trees

Observations: noisy DNA-seq trees

Ivanovic & El-Kebir

CNRein

Copy-number evolution in single cells

States: sets of CNV events per cell

Observations: single-cell DNA-seq

uses CNNaive for off-policy seeding

This paper **generalizes** the shared technique behind both into one framework for *any* discrete latent structure — with theory, off-policy theory, and baselines.

When should *you* reach for GReinSS?

Good fit ✓

- latent state is **discrete / combinatorial**
- you can **generate** it incrementally
- you know (or can compute) $\Pr(X \mid S)$
- observations are **indirect** & shared structure exists

Reach elsewhere ✗

- continuous latents → VAE / diffusion
- tractable exact E-step → classic EM
- rewards truly fixed & known → standard RL / GFlowNet

Recipe: ① write a generator for S · ② write $\Pr(X \mid S)$ · ③ `simpleTrainModel` · ④ `simpleInference` . *(optional)* add an off-policy proposal for hard instances.

Thank you — let's build

Code + notebook: `code/README.md`, `tutorial/GReinSS_demo.ipynb`

Recipe: generator for S + likelihood $\Pr(X \mid S) \rightarrow \text{train} \rightarrow \text{infer}$

Questions? · melkebir@illinois.edu